

(a) What are the main differences between training a FANN for a regression problem compared to a classification problem – in terms of network structure and cost function?

The main differences between training a Feedforward Artificial Neural Network (FANN) for a regression problem versus a classification problem lie predominantly in the output layer's structure and the choice of cost (loss) function.

- Network Structure (Output Layer):
 - Regression: The output layer typically consists of one single neuron if predicting a single continuous value. This neuron usually has a linear (or no) activation function, meaning its output can be any real number, directly corresponding to the predicted value. If predicting multiple continuous values (e.g., multiple coordinates), it would have one neuron per predicted value, still with linear activations.
 - Classification:
 - The output layer consists of one neuron for each class the model needs to distinguish (e.g., 2 neurons for a 2-class problem, 10 neurons for a 10-class problem like FashionMNIST).
 - The activation function on these output neurons is almost always Softmax. Softmax squashes the raw outputs (logits) into a probability distribution, where each output is between 0 and 1, and all outputs sum to 1. This allows interpreting the network's output as the probability of the input belonging to each class.
- Cost (Loss) Function:
 - Regression: The standard cost function is the Mean Squared Error (MSELoss). It calculates the average of the squared differences between the predicted continuous values and the true continuous values. This penalises larger errors more heavily and provides a continuous, differentiable landscape for optimization.
 - Classification: The standard cost function is Cross-Entropy Loss (nn.CrossEntropyLoss). This function is designed specifically for probability distributions. It measures the dissimilarity between the predicted probability distribution (from the Softmax layer) and the true probability distribution (represented by one-hot encoding). Critically, in

PyTorch, `nn.CrossEntropyLoss` internally applies the Softmax function, so the network's last layer should output raw logits, not probabilities. Cross-entropy provides strong gradients, especially when the model is confidently wrong, facilitating efficient learning.

(b) Why is it a good result when a 2-class classifier has a large AUC (area under the ROC curve)?

A large AUC (Area Under the Receiver Operating Characteristic curve) is an excellent result for a 2-class classifier because it signifies robust and strong discriminatory power across various classification thresholds.

Here's why:

1. Threshold-Independent Performance:

The ROC curve plots the True Positive Rate (TPR, or Probability of Detection/Sensitivity) against the False Positive Rate (FPR, or Probability of False Alarm) for all possible classification thresholds. This means AUC doesn't rely on a single, arbitrary decision threshold. A high AUC indicates good performance regardless of where the threshold is set.

2. Discriminatory Power:

- An AUC of 1.0 represents a perfect classifier that can distinguish between positive and negative classes with no errors. Its ROC curve would go straight up to (0,1) and then across to (1,1).
- An AUC of 0.5 represents a classifier no better than random guessing. Its ROC curve is the diagonal line from (0,0) to (1,1).
- A large AUC (closer to 1.0) means the model has a high probability of ranking a randomly chosen positive example higher than a randomly chosen negative example. It effectively separates the classes.

3. Robust to Imbalanced Datasets: In real-world scenarios, datasets are often imbalanced (e.g., many more "normal" cases than "abnormal" cases). Simple accuracy can be misleading in such cases. AUC, however, is a more reliable metric because it evaluates how well the model distinguishes between classes, independent of their prevalence. It's not biased by the majority class.

4. Trade-off Visualization: The ROC curve visually represents the trade-off between sensitivity and specificity. A large AUC implies that the classifier can achieve a high True Positive Rate without incurring an unacceptably high False

Positive Rate, which is often a critical balance in many applications (e.g., medical diagnostics, anomaly detection).

Therefore, a large AUC is a strong indicator of a classifier's overall quality and its ability to generalize effectively to unseen data in practical applications.

(c) Why is it good/promising when you can see that the accuracy for the training and test sets follows each other during/after training?

When the accuracy curves for both the training set and the test (or validation) set closely follow each other during and after training, it is a very good and promising result because it indicates strong generalization and absence of severe overfitting.

(d) What did you personally find interesting and/or useful an/or informative with the assignment, and why? If you actually did not find it enlightening, in that case we would like to know about why.

I found several aspects of this assignment particularly interesting and informative:

1. The Practical Nuances of PyTorch for Classification: While I had some exposure to FANNs for regression, directly implementing and debugging for classification in PyTorch (especially with `nn.CrossEntropyLoss` implicitly handling Softmax) was very insightful.
2. The Concrete Comparison of FANN vs. CNN for Images: Running both a FANN and a CNN on image classification (FashionMNIST) and seeing the tangible difference in test accuracy (e.g., FANN at 87.11% vs. CNN at 91.50%) was a powerful demonstration. It solidified my understanding of why CNNs are specialized and superior for images, reinforcing the concepts of spatial preservation and hierarchical feature learning.
3. The Importance of Sanity Checks and Hyperparameter Tuning: The debugging process for the noisy loss curve in the basic training loop (Problem 3.3 f) was very educational. Systematically trying different learning rates and switching to the Adam optimizer, and then immediately seeing the impact on the loss curve's stability, emphasized the practical iterative nature of deep learning development and the critical role of hyperparameter tuning. It showed how a simple plot can diagnose core issues.

(e) What did you find frustrating and/or difficult, and/or unnecessary, and why?

Hyperparameter Tuning (Learning Rate for SGD): Initially, getting the SGD optimizer to produce a smooth, decreasing loss curve with an appropriate learning rate was quite frustrating. The range of effective learning rates for SGD can be very narrow, and small deviations lead to chaotic or stagnant loss. While Adam proved more robust, the initial SGD struggles consumed a good amount of time and trial-and-error. This inherent difficulty of SGD is itself an informative lesson.

(f) What would you suggest as an improvement of this assignment for next year, and why?

- Structured Hyperparameter Tuning Exploration: Instead of just suggesting learning rates, it would be nice if there were a small sub-problem to experiment with optimizers and learning rates.
- Brief Introduction to Regularization: Given the observed overfitting in the FANN on FashionMNIST, a small optional section or tip on how to address overfitting would be an excellent extension as well.

(g) Did you spend more than 20 hours to complete this exercise? Why?

Yes, I spent approximately 25-30 hours to complete this exercise. The primary reasons for exceeding 20 hours were:

- Hyperparameter Tuning: The troubleshooting process for the noisy loss curve in Problem 3.3 (f) (experimenting with different learning rates and ultimately switching to Adam) took several iterations and re-runs, consuming a few hours.
- Deepening Understanding: I also spent additional time reviewing the linked videos and external resources to ensure a thorough understanding of concepts like CNN theory, parameter sharing, and the nuances of loss functions.